



Analyse Technique Profonde : Le Cerveau Adaptatif (algo.py)

Ce document dissèque la classe PortfolioBrain, une couche d'intelligence système qui transpose les mécanismes de bas niveau du protocole **TCP (RFC 6298)** vers la couche applicative. L'objectif est de résoudre le dilemme du timeout : être assez rapide pour l'UX, mais assez patient pour le réseau.

1. Constantes & Limites Physiques (Le Design System)

```
TIMEOUT_MIN = 1.0 # Plancher de sécurité (Anti-Bruit)
TIMEOUT_MAX = 10.0 # Plafond UX (Loi de Doherty)
MEMORY_TTL = 600 # Gestion du "Cold Start" (10 min)
```



Analyse Technique :

- **TIMEOUT_MIN (Le Bouclier)** : Sur le Free Tier de Render, votre code subit du *Throttling* (gel du CPU). Ce plancher de 1s agit comme un **filtre passe-bas** : il ignore les micro-gels du Cloud pour éviter de croire à une panne réseau alors que c'est juste le processeur qui "hoquette".
- **TIMEOUT_MAX (La Limite Physiologique)** : Basé sur la **Loi de Doherty**, l'attention de l'utilisateur décroche après 10s. En coupant la connexion à ce stade (Fail Fast), on libère aussi les workers Unicorn pour d'autres utilisateurs.

2. Initialisation & Sécurité des Threads

```
def __init__(self):
    self._lock = threading.Lock()
    self.srtt = self.DEFAULT_TIMEOUT # Moyenne pondérée
    self.rttvar = 0.5 # Incertitude (Jitter)
```



Analyse Technique :

- **threading.Lock() (Section Critique)** : Puisque Unicorn utilise des threads pour économiser la RAM (512Mo limit), ce verrou empêche les **Race Conditions**. Sans lui, deux requêtes simultanées pourraient corrompre les calculs statistiques.
- **Héritage TCP** : srtt représente "l'attente normale" et rttvar la mesure de la gigue. Un réseau instable fera exploser le rttvar, forçant l'algorithme à devenir plus prudent.

3. La Formule Magique (Van Jacobson, 1988)

```
def _calc_timeout_unsafe(self):
```

```
# Formule : Moyenne + 4 * Variation
rto = self.srtt + (4 * self.rttvar)
return max(self.TIMEOUT_MIN, min(self.TIMEOUT_MAX, rto))
```

Analyse Technique :

- **La Règle des 4 Sigmas** : C'est une application de l'**Inégalité de Chebyshev**. Dans un réseau parfait (Gaussien), 3 écarts-types suffiraient. Mais les latences réseaux ont une "longue traîne" (paquets très lents).
- **Le coefficient 4** : C'est l'heuristique robuste choisie pour couvrir 99.9% des scénarios, même les plus chaotiques, sans être trop pessimiste.

4. Stratégie de Réveil (Soft Decay)

```
if time_since_last > self.MEMORY_TTL:
    # On double l'incertitude (Prudence au réveil)
    self.rttvar = max(self.rttvar * 2, 1.0)
```

Analyse Technique :

- **Le problème de l'amnésie** : Si le serveur s'est endormi pendant 2 heures, les données en mémoire sont vieilles.
- **Approche Hybride** : On ne fait pas de reset total (pour garder l'historique du service), mais on double la marge de sécurité ($\text{rttvar} * 2$). C'est comme se souvenir du trajet habituel mais prévoir une marge énorme car on n'y est pas allé depuis longtemps.

5. L'Apprentissage Machine (Update)

Cas 1 : Échec (Algorithme de Karn)

```
if not success:
    self.current_timeout = min(self.TIMEOUT_MAX, self.current_timeout * 2)
```

- **"N'apprends jamais d'un échec"** : Si on intégrait le temps d'un timeout dans la moyenne, on fausserait tout. On applique un **Backoff Exponentiel** pour laisser le réseau respirer et éviter la *Congestion Collapse*.

Cas 2 : Succès (Algorithme de Jacobson)

```
diff = observed_latency - self.srtt
self.srtt = self.srtt + (0.125 * diff) # Alpha = 12.5%
self.rttvar = self.rttvar + (0.25 * (abs(diff) - self.rttvar)) # Beta = 25%
```

- **Lissage Alpha (0.125)** : Agit comme un amortisseur. Une seule requête lente ne fait pas exploser le timeout, le système reste stable face aux "spikes".

- **Lissage Beta (0.25)** : On apprend deux fois plus vite sur la variation que sur la moyenne pour réagir immédiatement si le réseau devient instable.

6. Observabilité : Briser la "Boîte Noire"

```
def get_stats(self):  
    return { "srtt": round(self.srtt, 3), ... }
```

Analyse Technique :

- En exposant ces données dans les logs, nous pouvons prouver factuellement si un incident est dû à une **instabilité réseau externe** (ex: panne AWS) ou à un bug du code Python. Cela transforme le "ça a planté" en une analyse d'ingénierie précise.

Ce fichier transforme une application web classique en un système organique capable de ressentir son environnement.